Mini Compilateur Pascal:

1: La grammaire :

PROGRAM ::= program ID ; BLOCK .
BLOCK ::= CONSTS VARS TABS INSTS
CONSTS ::= const ID = CONSTVAL { ,ID = CONSTVAL} ; | ε
CONSTVAL ::= STRING | REAL | NUM | BOOLEAN

VARS ::= var IDLIST
IDLIST ::= IDLIST | ID : TYPE ; | ε
TYPE ::= integer | string | real | boolean | char | array | record

PROCEDURES ::= { PROCEDURE_DEF }
FUNCTIONS ::= { FUNCTION_DEF }

PROCEDURE_DEF ::= procedure ID [(PARAM_LIST)] ; BLOCK ;
FUNCTION_DEF ::= function ID [(PARAM_LIST)] : TYPE ; BLOCK ;
PARAM_LIST ::= PARAM { ; PARAM }
PARAM ::= [var] ID { , ID } : TYPE *// 'var' optionnel indique passage par adresse*

ARRAY ::= array [ELEMENT { , ELEMENT }] of TYPE ;
RECORD ::= record { ID : TYPE; } end
ELEMENT ::= NUM | STRING | REAL | NUM : NUM
INSTS ::= begin INST { ; INST } end
INST ::= INSTS | AFFEC | SI | TANTQUE | ECRIRE | LIRE | FOR | CASE| ε
AFFEC ::= ID := EXPR

SI ::= if COND then INST { else IF_STATEMENT } | if COND then INST
IF_STATEMENT ::= if COND then INST { else IF_STATEMENT } | INST
REPEAT ::= repeat INSTRUCTION until CONDITION
FOR ::= for ID := EXPR to EXPR do INST | for ID := EXPR downto EXPR do INST
CASE ::= case EXPR of CASE_BRANCH {CASE_BRANCH ; } ; end
CASE_BRANCH ::= VALUE : INST
VALUE ::= NUM | STRING | BOOLEAN | ID

PROCEDURE_CALL ::= ID ([ARGUMENTS])
FUNCTION_CALL ::= ID ([ARGUMENTS])
ARGUMENTS ::= EXPRESSION { , EXPRESSION }

TANTQUE ::= while COND do INST
ECRIRE ::= write (EXPR { , EXPR })

LIRE ::= read (ID { , ID })
COND ::= EXPR RELOP EXPR
RELOP ::= = | <> | < | > | <= | >=
EXPR ::= TERM { ADDOP TERM }
ADDOP ::= + | -
TERM ::= FACT { MULOP FACT }
MULOP ::= * | /
FACT ::= ID | NUM | STRING | REAL | Boolean | Array_access | (EXPR)
Array_access ::= ID[NUM]
BOOL ::= true | false
STRING ::= ' char{char} '
ID ::= Lettre { Lettre | Chiffre }
NUM ::= Chiffre { Chiffre }
REAL ::= Chiffre { Chiffre } . Chiffre { Chiffre }
Chiffre ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
char ::= a|b|...|z|A|B|...|Z|0| 1|...|9| special_character

2: Règles sémantiques :

- **Règle 0:**
  Lors d'une affectation, il est impératif que les deux types soient compatibles :
  a := 42 ;
  b := a ;
  tab[4] := b;
  c := tab[2];
- **Règle 1:**
  Un entier peut être assigné à une variable de type flottant. De même, une variable réelle peut être comparée à un entier :
  IF a = 10
  a := 5.5
- **Règle 2:**
  Dans une instruction CASE, le type de l'expression doit être compatible avec les valeurs à comparer :
  CASE color OF
  'red': value := 30;
  'blue': value := 20;
- **Règle 3:**
  L'indice de boucle FOR doit être de type entier :
  FOR i := 1 TO 10 DO sum := sum + i;
- **Règle 4:**
  Les types doivent être compatibles dans l'expression de la condition du WHILE :
  WHILE score = 100
- **Règle 5:**

Compatibilité des types dans l'expression de la condition du IF :
IF age = 18

- **Règle 6:**
  Pour les instructions d'accès à un tableau comme tab[4] := b; ou c := tab[2];, l'indice ne doit pas dépasser la taille du tableau.
- **Règle 7:**
  Compatibilité des types dans EXPRESSION :
  total := total - 10;
  result := m + n;

3: Les Mnémoniques (interpréteur) :

- **ADD** : Additionne le sous-sommet et le sommet de la pile, puis place le résultat au sommet (similaire pour SUB, MUL, DIV).
- **EQL** : Place 1 au sommet de la pile si le sous-sommet est égal au sommet, sinon place 0 (similaire pour NEQ, GTR, LSS, GEQ, LEQ).
- **PRN** : Imprime la valeur au sommet de la pile, puis dépile.
- **INN** : Lit un entier de l'entrée, le stocke à l'adresse au sommet de la pile, puis dépile.
- **INT c** : Modifie le pointeur de pile en l'incrémentant de la constante c (c peut être négative).
- **LDI v** : Empile la valeur v sur la pile.
- **LDA a** : Empile l'adresse a sur la pile.
- **LDV** : Remplace la valeur au sommet par celle trouvée à l'adresse indiquée par le sommet (déréférence).
- **STO** : Stocke la valeur du sommet à l'adresse indiquée par le sous-sommet, puis dépile deux fois.
- **BRN i** : Effectue un saut inconditionnel vers l'instruction i.
- **BZE i** : Effectue un saut vers l'instruction i si la valeur au sommet est 0, puis dépile.
- **LEN i** : Calcule la longueur d'un tableau ou d'une chaîne de caractères et place le résultat au sommet de la pile.
- **FLDA a** : Charge l'adresse d'une constante flottante sur la pile.
- **SLDA a** : Charge l'adresse d'une chaîne de caractères sur la pile.
- **BLDA a** : Charge l'adresse d'une constante booléenne sur la pile.
- **FLDI v** : Charge une constante flottante v sur la pile.
- **SLDI v** : Charge une constante de type chaîne de caractères v sur la pile.
- **BLDI v** : Charge une constante booléenne v sur la pile.
- **TLDA a** : Charge l'adresse d'un type a sur la pile.
- **ILDA a** : Charge l'adresse d'une constante entière a sur la pile.
- **ILDI v** : Charge une constante entière v sur la pile.
- **EFOR i** : Marque le début d'une boucle `for` avec l'index i.
- **BFOR i** : Marque la fin d'une boucle `for` avec l'index i.

- **TABA i** : Déclare un tableau avec l'index i.
- **TABI i** : Déclare un index de tableau i.
- **CASET i** : Marque le début d'une section de cas dans une structure `case` avec l'index i.
- **CASEE i** : Marque la fin d'une section de cas dans une structure `case` avec l'index i.
- **HLT** : Arrête l'exécution.